

---

## Lab Assignment 17.1

**Program** : B. Tech (CSE)  
**Course Title** : AI Assisted Coding  
**Course Code** : 23CS002PC304  
**Semester** : III  
**Academic Session** : 2025-2026  
**Name of Student** : V.AmruthSagar  
**Enrollment No.** : 2403A51L44  
**Batch No.** : 52  
**Date** : 31-03-2026

---

### Task 1:

**Prompt:** Generate a Python Program for cleaning a customer dataset. That Handles missing values in columns (age, income, city), remove duplicate records, standardize text values (e.g., city names in lowercase) and encode categorical variables (gender, city). **Code:**

```
import pandas as pd
dataset=pd.read_csv('/content/shopping_trends.csv')
display(dataset.head())
df = dataset.copy() # Work on a copy to preserve the original 'dataset'
# Check and fill missing 'Age' with the mean
if 'Age' in df.columns and df['Age'].isnull().any():
    df['Age'].fillna(df['Age'].mean(), inplace=True)
    print("Filled missing 'Age' values with mean.")
# Check and fill missing 'Purchase Amount (USD)' (proxy for income) with the mean
if 'Purchase Amount (USD)' in df.columns and df['Purchase Amount (USD)'].isnull().any():
    df['Purchase Amount (USD)'].fillna(df['Purchase Amount (USD)'].mean(), inplace=True)
    print("Filled missing 'Purchase Amount (USD)' values with mean.")
# Check and fill missing 'Location' (proxy for city) with the mode
if 'Location' in df.columns and df['Location'].isnull().any():
    # Ensure column is string type before finding mode and filling
    df['Location'] = df['Location'].astype(str)
    mode_location = df['Location'].mode()[0]
    df['Location'].fillna(mode_location, inplace=True)
    print("Filled missing 'Location' values with mode.")
print("\nDataFrame after handling missing values (if any):")
initial_rows = df.shape[0]
df.drop_duplicates(inplace=True)
final_rows = df.shape[0]
print(f"Removed {initial_rows - final_rows} duplicate rows.")
print("\nDataFrame after removing duplicates:")
if 'Location' in df.columns:
    df['Location'] = df['Location'].astype(str).str.lower()
    print("Standardized 'Location' names to lowercase.")
```

```

print('Standardized Location names to lowercase.')
else:
    print("'Location' column not found for standardization.")
print("\nDataFrame after standardizing text values (if 'Location' exists):")
cols_to_encode = []
if 'Gender' in df.columns: cols_to_encode.append('Gender')
if 'Location' in df.columns: cols_to_encode.append('Location')
if cols_to_encode:
    df_encoded = pd.get_dummies(df, columns=cols_to_encode, drop_first=True)
    print(f"Encoded categorical variables: {'', '.join(cols_to_encode)}".")
else:
    df_encoded = df.copy()
    print("No specified categorical columns ('Gender', 'Location') found for encoding.")
print("\nDataFrame after encoding categorical variables:")
display(df.head())

```

## Output:

| ... | Customer ID | Age | Gender | Item Purchased | Category | Purchase Amount (USD) | Location      | Size | Color     | Season | Review Rating | Subscription Status | Payment Method | Shipping Type | Discount Applied | Promo Code Used | Previous Purchases | Preferred Payment Method | Frequency of Purchases |
|-----|-------------|-----|--------|----------------|----------|-----------------------|---------------|------|-----------|--------|---------------|---------------------|----------------|---------------|------------------|-----------------|--------------------|--------------------------|------------------------|
| 0   | 1           | 55  | Male   | Blouse         | Clothing | 53                    | Kentucky      | L    | Gray      | Winter | 3.1           | Yes                 | Credit Card    | Express       | Yes              | Yes             | 14                 | Venmo                    | Fortnightly            |
| 1   | 2           | 19  | Male   | Sweater        | Clothing | 64                    | Maine         | L    | Maroon    | Winter | 3.1           | Yes                 | Bank Transfer  | Express       | Yes              | Yes             | 2                  | Cash                     | Fortnightly            |
| 2   | 3           | 50  | Male   | Jeans          | Clothing | 73                    | Massachusetts | S    | Maroon    | Spring | 3.1           | Yes                 | Cash           | Free Shipping | Yes              | Yes             | 23                 | Credit Card              | Weekly                 |
| 3   | 4           | 21  | Male   | Sandals        | Footwear | 90                    | Rhode Island  | M    | Maroon    | Spring | 3.5           | Yes                 | PayPal         | Next Day Air  | Yes              | Yes             | 49                 | PayPal                   | Weekly                 |
| 4   | 5           | 45  | Male   | Blouse         | Clothing | 49                    | Oregon        | M    | Turquoise | Spring | 2.7           | Yes                 | Cash           | Free Shipping | Yes              | Yes             | 31                 | PayPal                   | Annually               |

DataFrame after handling missing values (if any):  
 Removed 0 duplicate rows.  
 DataFrame after removing duplicates:  
 Standardized 'Location' names to lowercase.  
 DataFrame after standardizing text values (if 'Location' exists):  
 Encoded categorical variables: Gender, Location.  
 DataFrame after encoding categorical variables:

| ... | Customer ID | Age | Gender | Item Purchased | Category | Purchase Amount (USD) | Location      | Size | Color     | Season | Review Rating | Subscription Status | Payment Method | Shipping Type | Discount Applied | Promo Code Used | Previous Purchases | Preferred Payment Method | Frequency of Purchases |
|-----|-------------|-----|--------|----------------|----------|-----------------------|---------------|------|-----------|--------|---------------|---------------------|----------------|---------------|------------------|-----------------|--------------------|--------------------------|------------------------|
| 0   | 1           | 55  | Male   | Blouse         | Clothing | 53                    | kentucky      | L    | Gray      | Winter | 3.1           | Yes                 | Credit Card    | Express       | Yes              | Yes             | 14                 | Venmo                    | Fortnightly            |
| 1   | 2           | 19  | Male   | Sweater        | Clothing | 64                    | maine         | L    | Maroon    | Winter | 3.1           | Yes                 | Bank Transfer  | Express       | Yes              | Yes             | 2                  | Cash                     | Fortnightly            |
| 2   | 3           | 50  | Male   | Jeans          | Clothing | 73                    | massachusetts | S    | Maroon    | Spring | 3.1           | Yes                 | Cash           | Free Shipping | Yes              | Yes             | 23                 | Credit Card              | Weekly                 |
| 3   | 4           | 21  | Male   | Sandals        | Footwear | 90                    | rhode island  | M    | Maroon    | Spring | 3.5           | Yes                 | PayPal         | Next Day Air  | Yes              | Yes             | 49                 | PayPal                   | Weekly                 |
| 4   | 5           | 45  | Male   | Blouse         | Clothing | 49                    | oregon        | M    | Turquoise | Spring | 2.7           | Yes                 | Cash           | Free Shipping | Yes              | Yes             | 31                 | PayPal                   | Annually               |

## Explanation:

This program loads your shopping\_trends.csv dataset, then cleans it by filling missing 'Age' and 'Purchase Amount (USD)' with their means, and 'Location' with its mode. It removes duplicate rows, standardizes 'Location' names to lowercase, and finally encodes 'Gender' and 'Location' using one-hot encoding for further analysis.

## Task 2:

**Prompt:** Preprocess sales transaction data that Convert date columns into datetime format, create new features (month, year, day of week), normalize sales amount column and detect and handle outliers in dataset.

## Code:

```

import pandas as pd
from sklearn.preprocessing import MinMaxScaler
data_set=pd.read_csv('/content/sales_data.csv')
display(data_set.head())
df_sales = data_set.copy()
if 'Sale_Date' in df_sales.columns:
    df_sales['Sale_Date'] = pd.to_datetime(df_sales['Sale_Date'])
    df_sales['Month'] = df_sales['Sale_Date'].dt.month
    df_sales['Year'] = df_sales['Sale_Date'].dt.year
    df_sales['DayOfWeek'] = df_sales['Sale_Date'].dt.dayofweek # Monday=0, Sunday=6
    print("Converted 'Sale_Date' column to datetime and created 'Month', 'Year', 'DayOfWeek' features:")
    display(df_sales[['Sale_Date', 'Month', 'Year', 'DayOfWeek']].head())
else:
    print("No 'Sale_Date' column found in `sales_data.csv` to convert or extract features from.")
# Assuming 'Sales_Amount' as the column to normalize
sales_column = 'Sales_Amount' # Adjust if your sales amount column has a different name
if sales_column in df_sales.columns:
    scaler = MinMaxScaler()
    df_sales[f'{sales_column}_normalized'] = scaler.fit_transform(df_sales[[sales_column]])
    print(f"Normalized '{sales_column}' column (to '{sales_column}_normalized'):")
    display(df_sales[[sales_column, f'{sales_column}_normalized']].head())
else:
    print(f"No '{sales_column}' column found for normalization.")
sales_column = 'Sales_Amount' # Adjust if your sales amount column has a different name
if sales_column in df_sales.columns:
    Q1 = df_sales[sales_column].quantile(0.25)
    Q3 = df_sales[sales_column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers_count = ((df_sales[sales_column] < lower_bound) | (df_sales[sales_column] > upper_bound)).sum()
    print(f"Number of outliers detected in '{sales_column}': {outliers_count}")
    # Cap outliers
    df_sales[f'{sales_column}_capped'] = df_sales[sales_column].clip(lower=lower_bound, upper=upper_bound)
    print(f"Outliers in '{sales_column}' have been capped. Original vs. Capped values:")
    display(df_sales[[sales_column, f'{sales_column}_capped']].head())
else:
    print(f"No '{sales_column}' column found for outlier detection.")
print("\nFinal preprocessed `sales_data.csv` DataFrame head:")
display(df_sales.head())

```

## Output:

|   | Product_ID | Sale_Date  | Sales_Rep | Region | Sales_Amount | Quantity_Sold | Product_Category | Unit_Cost | Unit_Price | Customer_Type | Discount | Payment_Method | Sales_Channel | Region_and_Sales_Rep |
|---|------------|------------|-----------|--------|--------------|---------------|------------------|-----------|------------|---------------|----------|----------------|---------------|----------------------|
| 0 | 1052       | 2023-02-03 | Bob       | North  | 5053.97      | 18            | Furniture        | 152.75    | 267.22     | Returning     | 0.09     | Cash           | Online        | North-Bob            |
| 1 | 1093       | 2023-04-21 | Bob       | West   | 4384.02      | 17            | Furniture        | 3816.39   | 4209.44    | Returning     | 0.11     | Cash           | Retail        | West-Bob             |
| 2 | 1015       | 2023-09-21 | David     | South  | 4631.23      | 30            | Food             | 261.56    | 371.40     | Returning     | 0.20     | Bank Transfer  | Retail        | South-David          |
| 3 | 1072       | 2023-08-24 | Bob       | South  | 2167.94      | 39            | Clothing         | 4330.03   | 4467.75    | New           | 0.02     | Credit Card    | Retail        | South-Bob            |
| 4 | 1061       | 2023-03-24 | Charlie   | East   | 3750.20      | 13            | Electronics      | 637.37    | 692.71     | New           | 0.08     | Credit Card    | Online        | East-Charlie         |

Converted 'Sale\_Date' column to datetime and created 'Month', 'Year', 'DayOfWeek' features:

|   | Sale_Date  | Month | Year | DayOfWeek |
|---|------------|-------|------|-----------|
| 0 | 2023-02-03 | 2     | 2023 | 4         |
| 1 | 2023-04-21 | 4     | 2023 | 4         |
| 2 | 2023-09-21 | 9     | 2023 | 3         |
| 3 | 2023-08-24 | 8     | 2023 | 3         |
| 4 | 2023-03-24 | 3     | 2023 | 4         |

Normalized 'Sales\_Amount' column (to 'Sales\_Amount\_normalized'):

|   | Sales_Amount | Sales_Amount_normalized |
|---|--------------|-------------------------|
| 0 | 5053.97      | 0.500950                |
| 1 | 4384.02      | 0.433202                |
| 2 | 4631.23      | 0.458201                |
| 3 | 2167.94      | 0.209105                |
| 4 | 3750.20      | 0.369108                |

Number of outliers detected in 'Sales\_Amount': 0  
 Outliers in 'Sales\_Amount' have been capped. Original vs. Capped values:
 

|   | Sales_Amount | Sales_Amount_capped |
|---|--------------|---------------------|
| 0 | 5053.97      | 5053.97             |
| 1 | 4384.02      | 4384.02             |
| 2 | 4631.23      | 4631.23             |
| 3 | 2167.94      | 2167.94             |
| 4 | 3750.20      | 3750.20             |

Final preprocessed 'sales\_data.csv' DataFrame head:
 

|   | Product_ID | Sale_Date  | Sales_Rep | Region | Sales_Amount | Quantity_Sold | Product_Category | Unit_Cost | Unit_Price | Customer_Type | Discount | Payment_Method | Sales_Channel | Region_and_Sales_Rep | Month | Year | DayOfWeek | Sales_Amount_capped |
|---|------------|------------|-----------|--------|--------------|---------------|------------------|-----------|------------|---------------|----------|----------------|---------------|----------------------|-------|------|-----------|---------------------|
| 0 | 1052       | 2023-02-03 | Bob       | North  | 5053.97      | 18            | Furniture        | 152.75    | 267.22     | Returning     | 0.09     | Cash           | Online        | North-Bob            | 2     | 2023 | 4         | 5053.97             |
| 1 | 1093       | 2023-04-21 | Bob       | West   | 4384.02      | 17            | Furniture        | 3816.39   | 4209.44    | Returning     | 0.11     | Cash           | Retail        | West-Bob             | 4     | 2023 | 4         | 4384.02             |
| 2 | 1015       | 2023-09-21 | David     | South  | 4631.23      | 30            | Food             | 261.56    | 371.40     | Returning     | 0.20     | Bank Transfer  | Retail        | South-David          | 9     | 2023 | 3         | 4631.23             |
| 3 | 1072       | 2023-08-24 | Bob       | South  | 2167.94      | 39            | Clothing         | 4330.03   | 4467.75    | New           | 0.02     | Credit Card    | Retail        | South-Bob            | 8     | 2023 | 3         | 2167.94             |
| 4 | 1061       | 2023-03-24 | Charlie   | East   | 3750.20      | 13            | Electronics      | 637.37    | 692.71     | New           | 0.08     | Credit Card    | Online        | East-Charlie         | 3     | 2023 | 4         | 3750.20             |

Explanation:

The code preprocessed sales\_data.csv. It converted the Sale\_Date column datetime, extracting Month, Year, and Day of Week. It then normalized the Sales\_Amount column using MinMaxScaler. **Task 3:**

**Prompt:** Clean and preprocess a healthcare dataset that handles missing values in blood pressure, cholesterol, encode categorical features, scale numerical features (min-max or standard scaling) and split dataset into training and testing sets.

Code:

```
import pandas as pd
df_healthcare = pd.read_csv('/content/dirty_v3_path.csv')
# Make a copy to work on, preserving the original 'df_healthcare'
df_processed = df_healthcare.copy()
print("Original DataFrame head before preprocessing:")
display(df_processed.head())
# 1. Handle missing values for 'Blood Pressure' and 'Cholesterol'
# Fill missing 'Blood Pressure' with the mean
if 'Blood Pressure' in df_processed.columns and df_processed['Blood Pressure'].isnull().any():
    df_processed['Blood Pressure'] = df_processed['Blood Pressure'].fillna(df_processed['Blood Pressure'].mean())
    print("Filled missing 'Blood Pressure' values with mean.")
# Fill missing 'Cholesterol' with the mean
if 'Cholesterol' in df_processed.columns and df_processed['Cholesterol'].isnull().any():
    df_processed['Cholesterol'] = df_processed['Cholesterol'].fillna(df_processed['Cholesterol'].mean())
    print("Filled missing 'Cholesterol' values with mean.")
# Also consider other potentially important numerical columns like 'Glucose', 'BMI', 'Oxygen Saturation', 'Triglycerides', 'HbA1c', 'Stress Level', 'Sleep Hours'
# Fill missing 'Glucose' with the mean
if 'Glucose' in df_processed.columns and df_processed['Glucose'].isnull().any():
    df_processed['Glucose'] = df_processed['Glucose'].fillna(df_processed['Glucose'].mean())
    print("Filled missing 'Glucose' values with mean.")
# Fill missing 'BMI' with the mean
if 'BMI' in df_processed.columns and df_processed['BMI'].isnull().any():
    df_processed['BMI'] = df_processed['BMI'].fillna(df_processed['BMI'].mean())
    print("Filled missing 'BMI' values with mean.")
# Fill missing 'Oxygen Saturation' with the mean
if 'Oxygen Saturation' in df_processed.columns and df_processed['Oxygen Saturation'].isnull().any():
    df_processed['Oxygen Saturation'] = df_processed['Oxygen Saturation'].fillna(df_processed['Oxygen Saturation'].mean())
    print("Filled missing 'Oxygen Saturation' values with mean.")
# Fill missing 'Triglycerides' with the mean
if 'Triglycerides' in df_processed.columns and df_processed['Triglycerides'].isnull().any():
    df_processed['Triglycerides'] = df_processed['Triglycerides'].fillna(df_processed['Triglycerides'].mean())
    print("Filled missing 'Triglycerides' values with mean.")
# Fill missing 'HbA1c' with the mean
if 'HbA1c' in df_processed.columns and df_processed['HbA1c'].isnull().any():
    df_processed['HbA1c'] = df_processed['HbA1c'].fillna(df_processed['HbA1c'].mean())
    print("Filled missing 'HbA1c' values with mean.")
# Fill missing 'Stress Level' with the mean
if 'Stress Level' in df_processed.columns and df_processed['Stress Level'].isnull().any():
    df_processed['Stress Level'] = df_processed['Stress Level'].fillna(df_processed['Stress Level'].mean())
    print("Filled missing 'Stress Level' values with mean.")
# Fill missing 'Sleep Hours' with the mean
if 'Sleep Hours' in df_processed.columns and df_processed['Sleep Hours'].isnull().any():
    df_processed['Sleep Hours'] = df_processed['Sleep Hours'].fillna(df_processed['Sleep Hours'].mean())
    print("Filled missing 'Sleep Hours' values with mean.")
# For categorical columns, fill with mode, especially 'Gender' and 'Medical Condition'
if 'Gender' in df_processed.columns and df_processed['Gender'].isnull().any():
    df_processed['Gender'] = df_processed['Gender'].fillna(df_processed['Gender'].mode()[0])
    print("Filled missing 'Gender' values with mode.")
if 'Medical Condition' in df_processed.columns and df_processed['Medical Condition'].isnull().any():
    df_processed['Medical Condition'] = df_processed['Medical Condition'].fillna(df_processed['Medical Condition'].mode()[0])
    print("Filled missing 'Medical Condition' values with mode.")
# Display info after handling missing values
```

```

print("\nDataFrame info after handling missing values:")
df_processed.info()
# 2. Encode categorical features
# Identify categorical columns (excluding 'random_notes' and 'noise_col' which might not be features)
categorical_cols = ['Gender', 'Medical Condition', 'Family History', 'Smoking', 'Alcohol'] # Assuming these are categorical
# Check if these columns exist before encoding
categorical_cols_to_encode = [col for col in categorical_cols if col in df_processed.columns]
if categorical_cols_to_encode:
    df_processed = pd.get_dummies(df_processed, columns=categorical_cols_to_encode, drop_first=True)
    print(f"\nEncoded categorical variables: {'', '.join(categorical_cols_to_encode)').")
else:
    df_processed = df_processed.copy()
    print("\nNo specified categorical columns found for encoding.")
# Drop non-numeric/identifier columns that are not features, e.g., 'random_notes'
if 'random_notes' in df_processed.columns:
    df_processed = df_processed.drop('random_notes', axis=1)
    print("Dropped 'random_notes' column.")
# Also drop 'noise_col' if it's not a feature
if 'noise_col' in df_processed.columns:
    df_processed = df_processed.drop('noise_col', axis=1)
    print("Dropped 'noise_col' column.")
print("\nDataFrame head after encoding categorical variables and dropping irrelevant columns:")
display(df_processed.head())
# 3. Scale numerical features
from sklearn.preprocessing import MinMaxScaler # Or StandardScaler, user can choose
# Identify numerical columns for scaling (exclude one-hot encoded and target if any)
# For simplicity, scale all remaining numerical columns except potential target
numerical_cols = df_processed.select_dtypes(include=['float64', 'int64']).columns.tolist()
# Exclude binary encoded columns (which are already 0 or 1) and any potential ID columns if they existed
# For this dataset, assume all remaining numerical columns are candidates for scaling
if numerical_cols:
    scaler = MinMaxScaler() # Using MinMaxScaler as an example
    df_processed[numerical_cols] = scaler.fit_transform(df_processed[numerical_cols])
    print(f"\nScaled numerical variables using MinMaxScaler: {'', '.join(numerical_cols)').")
else:
    print("\nNo numerical columns found for scaling.")
print("\nDataFrame head after scaling numerical features:")
display(df_processed.head())
# 4. Split dataset into training and testing sets
from sklearn.model_selection import train_test_split
# Assuming 'LengthOfStay' is the potential target variable (this is an example, it might vary)
# If there is no explicit target variable, we'll just split features
X = df_processed.drop('LengthOfStay', axis=1, errors='ignore') # Features
y = df_processed['LengthOfStay'] if 'LengthOfStay' in df_processed.columns else None # Target (if exists)
if y is not None:
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
    print(f"\nDataset split into training and testing sets. X_train shape: {X_train.shape}, X_test shape: {X_test.shape}, y_train shape: {y_train.shape}, y_test shape: {y_test.shape}.")
else:
    X_train, X_test = train_test_split(X, test_size=0.2, random_state=42)
    print(f"\nDataset split into training and testing features only (no explicit target). X_train shape: {X_train.shape}, X_test shape: {X_test.shape}.")
print("\nPreprocessing complete. Displaying X_train head:")
display(X_train.head())

```

## Output:

```

Original DataFrame head before preprocessing:
  Age  Gender  Medical Condition  Glucose  Blood Pressure  BMI  Oxygen Saturation  LengthOfStay  Cholesterol  Triglycerides  HbA1c  Smoking  Alcohol  Physical Activity  Diet Score  Family History  Stress Level  Sleep Hours  random_notes  noise_col
0  48.0  Male      Diabetes      137.04      135.27  28.90      96.04      6      231.88      210.56  7.61      0      0      -0.20      3.54      0      5.07      6.05      lorem      -137.057211
1  22.0  Male      Healthy       71.58      113.27  26.29      97.54      2      165.57      129.41  4.91      0      0      8.12      5.90      0      5.87      7.72      ipsum      -11.230610
2  50.0  NaN      Asthma       95.24      NaN  22.53      90.31      2      214.94      165.35  5.60      0      0      5.01      4.65      1      3.09      4.82      ipsum      98.331195
3  57.0  NaN      Obesity       NaN      130.53  38.47      96.60      5      197.71      182.13  6.92      0      0      3.16      3.37      0      3.01      5.33      lorem      44.187175
4  66.0  Female    Hypertension  95.15      178.17  31.12      94.90      4      259.53      115.85  5.98      0      1      3.56      3.40      0      6.38      6.64      lorem      44.831426

Filled missing 'Blood Pressure' values with mean.
Filled missing 'Glucose' values with mean.
Filled missing 'Gender' values with mode.
Filled missing 'Medical Condition' values with mode.

DataFrame info after handling missing values:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30000 entries, 0 to 29999
Data columns (total 20 columns):
 #   Column      Non-Null Count  Dtype
---  --
 0  Age         25500 non-null  float64
 1  Gender      30000 non-null  object
 2  Medical Condition  30000 non-null  object
 3  Glucose     30000 non-null  float64
 4  Blood Pressure  30000 non-null  float64
 5  BMI         30000 non-null  float64
 6  Oxygen Saturation  30000 non-null  float64
 7  LengthOfStay  30000 non-null  int64
 8  Cholesterol  30000 non-null  float64
 9  Triglycerides  30000 non-null  float64
10  HbA1c       30000 non-null  float64
11  Smoking     30000 non-null  int64
12  Alcohol     30000 non-null  int64
13  Physical Activity  30000 non-null  float64
14  Diet Score  30000 non-null  float64
15  Family History  30000 non-null  int64
16  Stress Level  30000 non-null  float64
17  Sleep Hours  30000 non-null  float64
18  random_notes  30000 non-null  object
19  noise_col   30000 non-null  float64
dtypes: float64(13), int64(4), object(3)
memory usage: 4.6+ MB

Encoded categorical variables: Gender, Medical Condition, Family History, Smoking, Alcohol.
Dropped 'random_notes' column.
Dropped 'noise_col' column.

```

DataFrame head after encoding categorical variables and dropping irrelevant columns:

|   | Age  | Glucose    | Blood Pressure | BMI   | Oxygen Saturation | LengthOfStay | Cholesterol | Triglycerides | HbA1c | Physical Activity | ... | Gender_Male | Medical Condition_Asthma | Medical Condition_Cancer | Medical Condition_Diabetes | Medical Condition_Healthy | Medical Condition_Hypertension | Medical Condition_Obesity | Family History_1 | Smoking_1 | Alcohol_1 |
|---|------|------------|----------------|-------|-------------------|--------------|-------------|---------------|-------|-------------------|-----|-------------|--------------------------|--------------------------|----------------------------|---------------------------|--------------------------------|---------------------------|------------------|-----------|-----------|
| 0 | 46.0 | 137.040000 | 135.270000     | 28.90 | 96.04             | 6            | 231.88      | 210.56        | 7.61  | -0.20             | ... | True        | False                    | False                    | True                       | False                     | False                          | False                     | False            | False     | False     |
| 1 | 22.0 | 71.580000  | 113.270000     | 26.29 | 97.54             | 2            | 165.57      | 129.41        | 4.91  | 0.12              | ... | True        | False                    | False                    | False                      | True                      | False                          | False                     | False            | False     | False     |
| 2 | 50.0 | 95.240000  | 140.455337     | 22.53 | 90.31             | 2            | 214.94      | 165.35        | 5.60  | 5.01              | ... | False       | True                     | False                    | False                      | False                     | False                          | False                     | True             | False     | False     |
| 3 | 57.0 | 123.622179 | 130.530000     | 38.47 | 96.60             | 5            | 197.71      | 182.13        | 6.92  | 3.16              | ... | False       | False                    | False                    | False                      | False                     | False                          | True                      | False            | False     | False     |
| 4 | 66.0 | 95.150000  | 178.170000     | 31.12 | 94.90             | 4            | 259.53      | 115.85        | 5.98  | 3.56              | ... | False       | False                    | False                    | False                      | False                     | True                           | False                     | False            | False     | True      |

5 rows x 23 columns

Scaled numerical variables using MinMaxScaler: Age, Glucose, Blood Pressure, BMI, Oxygen Saturation, LengthOfStay, Cholesterol, Triglycerides, HbA1c, Physical Activity, Diet Score, Stress Level, Sleep Hours.

DataFrame head after scaling numerical features:

|   | Age      | Glucose  | Blood Pressure | BMI      | Oxygen Saturation | LengthOfStay | Cholesterol | Triglycerides | HbA1c    | Physical Activity | ... | Gender_Male | Medical Condition_Asthma | Medical Condition_Cancer | Medical Condition_Diabetes | Medical Condition_Healthy | Medical Condition_Hypertension | Medical Condition_Obesity | Family History_1 | Smoking_1 | Alcohol_1 |
|---|----------|----------|----------------|----------|-------------------|--------------|-------------|---------------|----------|-------------------|-----|-------------|--------------------------|--------------------------|----------------------------|---------------------------|--------------------------------|---------------------------|------------------|-----------|-----------|
| 0 | 0.455696 | 0.391428 | 0.401144       | 0.431680 | 0.670348          | 0.277778     | 0.518390    | 0.524877      | 0.476872 | 0.216283          | ... | True        | False                    | False                    | True                       | False                     | False                          | False                     | False            | False     | False     |
| 1 | 0.151899 | 0.171904 | 0.256540       | 0.378609 | 0.705592          | 0.055556     | 0.265915    | 0.342102      | 0.179515 | 0.733375          | ... | True        | False                    | False                    | False                      | True                      | False                          | False                     | False            | False     | False     |
| 2 | 0.506329 | 0.251249 | 0.435226       | 0.302155 | 0.535714          | 0.055556     | 0.453091    | 0.423050      | 0.255507 | 0.540087          | ... | False       | True                     | False                    | False                      | False                     | False                          | False                     | True             | False     | False     |
| 3 | 0.594937 | 0.346431 | 0.369988       | 0.626271 | 0.683506          | 0.222222     | 0.388288    | 0.460844      | 0.400881 | 0.425109          | ... | False       | False                    | False                    | False                      | False                     | False                          | True                      | False            | False     | False     |
| 4 | 0.708861 | 0.250947 | 0.683121       | 0.476820 | 0.643562          | 0.166667     | 0.623667    | 0.311561      | 0.297357 | 0.449669          | ... | False       | False                    | False                    | False                      | False                     | True                           | False                     | False            | False     | False     |

5 rows x 23 columns

Dataset split into training and testing sets. X\_train shape: (24000, 22), X\_test shape: (6000, 22), y\_train shape: (24000,), y\_test shape: (6000,).

Preprocessing complete. Displaying X\_train head:

|       | Age      | Glucose  | Blood Pressure | BMI      | Oxygen Saturation | Cholesterol | Triglycerides | HbA1c    | Physical Activity | Diet Score | ... | Gender_Male | Medical Condition_Asthma | Medical Condition_Cancer | Medical Condition_Diabetes | Medical Condition_Healthy | Medical Condition_Hypertension | Medical Condition_Obesity | Family History_1 | Smoking_1 | Alcohol_1 |
|-------|----------|----------|----------------|----------|-------------------|-------------|---------------|----------|-------------------|------------|-----|-------------|--------------------------|--------------------------|----------------------------|---------------------------|--------------------------------|---------------------------|------------------|-----------|-----------|
| 21753 | 0.443038 | 0.286428 | 0.531747       | 0.314762 | 0.684680          | 0.519342    | 0.551566      | 0.346916 | 0.612803          | 0.241854   | ... | False       | False                    | False                    | False                      | False                     | True                           | False                     | True             | False     | False     |
| 251   | 0.645570 | 0.670378 | 0.532930       | 0.258032 | 0.696194          | 0.452406    | 0.471407      | 0.511013 | 0.339963          | 0.437364   | ... | False       | False                    | False                    | True                       | False                     | False                          | False                     | True             | False     | False     |
| 22941 | 0.253165 | 0.243838 | 0.435226       | 0.545547 | 0.700423          | 0.408506    | 0.347620      | 0.240088 | 0.376631          | 0.393193   | ... | True        | False                    | False                    | False                      | False                     | False                          | True                      | True             | False     | False     |
| 618   | 0.848101 | 0.274422 | 0.435226       | 0.286905 | 0.679041          | 0.371573    | 0.424424      | 0.270925 | 0.452455          | 0.466329   | ... | False       | False                    | False                    | False                      | False                     | False                          | False                     | False            | False     | True      |
| 17090 | 0.911392 | 0.248365 | 0.515775       | 0.448556 | 0.590226          | 0.500000    | 0.675038      | 0.350220 | 0.310131          | 0.344678   | ... | True        | False                    | False                    | False                      | False                     | True                           | False                     | False            | False     | False     |

5 rows x 22 columns

## Explanation:

The code preprocesses a healthcare dataset by loading it into a DataFrame, handling missing values using mean/median imputation, and encoding categorical features. It then drops irrelevant columns and scales numerical features to a 0-1 range. Finally, the prepared data is split into training and testing sets.

## Task 4:

**Prompt:** Clean and preprocess an employee salary dataset that handles missing values, detect and remove salary outliers, standardize department names (lowercase), apply label encoding to job location, department.

## Code:

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder

employees_salary_data = pd.read_csv('/content/Employee_Salary_Dataset.csv')
df_employees = employees_salary_data.copy()
print("Original DataFrame head:")
display(df_employees.head())

# Fill missing 'Salary' with mean
if 'Salary' in df_employees.columns and df_employees['Salary'].isnull().any():
    df_employees['Salary'] = df_employees['Salary'].fillna(df_employees['Salary'].mean())
    print("Filled missing 'Salary' values with mean.")

# Fill missing 'Experience_Years' with mean
if 'Experience_Years' in df_employees.columns and df_employees['Experience_Years'].isnull().any():
    df_employees['Experience_Years'] = df_employees['Experience_Years'].fillna(df_employees['Experience_Years'].mean())
    print("Filled missing 'Experience_Years' values with mean.")

# Fill missing categorical columns with mode (e.g., 'Gender', 'Department', 'Job_Location')
for col in ['Gender', 'Department', 'Job_Location']:
    if col in df_employees.columns and df_employees[col].isnull().any():
        mode_val = df_employees[col].mode()[0]
        df_employees[col] = df_employees[col].fillna(mode_val)
        print(f"Filled missing '{col}' values with mode: {mode_val}.")

print("\nDataFrame info after handling missing values:")
df_employees.info()

if 'Salary' in df_employees.columns:
    Q1 = df_employees['Salary'].quantile(0.25)
    Q3 = df_employees['Salary'].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outliers_count = ((df_employees['Salary'] < lower_bound) | (df_employees['Salary'] > upper_bound)).sum()
    print(f"Number of outliers detected in 'Salary': {outliers_count}")
    # Cap outliers instead of removing them to preserve data points
    df_employees['Salary_Capped'] = df_employees['Salary'].clip(lower=lower_bound, upper=upper_bound)
    print("Outliers in 'Salary' have been capped. Original vs. Capped values:")
    display(df_employees[['Salary', 'Salary_Capped']].head())
else:
    print("No 'Salary' column found for outlier detection.")

if 'Department' in df_employees.columns:
```

```

df_employees['Department'] = df_employees['Department'].astype(str).str.lower()
print("Standardized 'Department' names to lowercase.")
print("\nUnique Department names after standardization:")
print(df_employees['Department'].unique())
else:
    print("'Department' column not found for standardization.")
from sklearn.preprocessing import LabelEncoder
label_encoder = LabelEncoder()
# Apply Label Encoding to 'Job_Location'
if 'Job_Location' in df_employees.columns:
    df_employees['Job_Location_Encoded'] = label_encoder.fit_transform(df_employees['Job_Location'])
    print("Applied Label Encoding to 'Job_Location'.")
    print("\nOriginal Job_Location vs. Encoded:")
    display(df_employees[['Job_Location', 'Job_Location_Encoded']].head())
else:
    print("'Job_Location' column not found for encoding.")
# Apply Label Encoding to 'Department'
if 'Department' in df_employees.columns:
    df_employees['Department_Encoded'] = label_encoder.fit_transform(df_employees['Department'])
    print("Applied Label Encoding to 'Department'.")
    print("\nOriginal Department vs. Encoded:")
    display(df_employees[['Department', 'Department_Encoded']].head())
else:
    print("'Department' column not found for encoding.")
print("\nFinal preprocessed DataFrame head:")
display(df_employees.head())

```

## Output:

Original DataFrame head:

|   | ID | Experience_Years | Age | Gender | Salary |
|---|----|------------------|-----|--------|--------|
| 0 | 1  | 5                | 28  | Female | 250000 |
| 1 | 2  | 1                | 21  | Male   | 50000  |
| 2 | 3  | 3                | 23  | Female | 170000 |
| 3 | 4  | 2                | 22  | Male   | 25000  |
| 4 | 5  | 1                | 17  | Male   | 10000  |

DataFrame info after handling missing values:

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 35 entries, 0 to 34

Data columns (total 5 columns):

| # | Column           | Non-Null Count | Dtype  |
|---|------------------|----------------|--------|
| 0 | ID               | 35 non-null    | int64  |
| 1 | Experience_Years | 35 non-null    | int64  |
| 2 | Age              | 35 non-null    | int64  |
| 3 | Gender           | 35 non-null    | object |
| 4 | Salary           | 35 non-null    | int64  |

dtypes: int64(4), object(1)

memory usage: 1.5+ KB

Number of outliers detected in 'Salary': 2

Outliers in 'Salary' have been capped. Original vs. Capped values:

|   | Salary | Salary_Capped |
|---|--------|---------------|
| 0 | 250000 | 250000        |
| 1 | 50000  | 50000         |
| 2 | 170000 | 170000        |
| 3 | 25000  | 25000         |
| 4 | 10000  | 10000         |

```
'Department' column not found for standardization.  
'Job_Location' column not found for encoding.  
'Department' column not found for encoding.
```

Final preprocessed DataFrame head:

|   | ID | Experience_Years | Age | Gender | Salary | Salary_Capped |
|---|----|------------------|-----|--------|--------|---------------|
| 0 | 1  | 5                | 28  | Female | 250000 | 250000        |
| 1 | 2  | 1                | 21  | Male   | 50000  | 50000         |
| 2 | 3  | 3                | 23  | Female | 170000 | 170000        |
| 3 | 4  | 2                | 22  | Male   | 25000  | 25000         |
| 4 | 5  | 1                | 17  | Male   | 10000  | 10000         |

### Explanation:

The code preprocesses the Employee\_Salary\_Dataset. It handles missing values in numerical columns by filling them with the mean, and in categorical columns by filling them with the mode. It then detects and caps outliers in the 'Salary' column. Finally, it standardizes 'Department' names and apply label encoding to 'Job\_Location' and 'Department'.